

Student Schedule Management System

Written Report

Object-Oriented Programming Course Project (JC1503)

CS Group 11

South China Normal University

Group Members and Contributions:

Name	Primary Contributions
CHENGJIN DU	Schedule service, SCNU scraper, BST implementation
HONGRUI LIU	Knowledge module (RAG, vector store), LLM integration
JIEXUN WEI	Frontend development (Vue 2), knowledge graph visualization
LINGXIAO WANG	Auth service, hash table implementation, security design
TIANLI ZHU	Note service, text chunking, SQLite storage layer
YIXUAN WANG	Queue/linked list implementation, testing, logging system

May 2026

Word Count: approximately 4,200

1 Problem and System Design

1.1 Real-World Context and Problem Statement

University students at South China Normal University face daily challenges in managing their academic schedules. Common practices—paper timetables, smartphone screenshots, or generic calendar applications—suffer from three fundamental limitations: they lack integration with the university’s live academic portal, offer no intelligent features for course material organisation, and require manual synchronisation whenever schedules change.

Our system addresses these problems by providing a purpose-built web application that:

1. Fetches live schedule data directly from the SCNU academic portal (Qiangzhi system) using the student’s own credentials.
2. Supports full CRUD operations for manual schedule management.
3. Provides real-time queries answering "what course is happening now?"
4. Integrates a knowledge workspace where students upload notes, search semantically across all materials, ask AI-powered questions grounded in their own documents, and explore conceptual relationships through an interactive knowledge graph.

This is not a collection of isolated utilities but a coherent system where schedule management and knowledge organisation reinforce each other: a student checks their current course, then immediately accesses the corresponding knowledge workspace to review or upload notes.

1.2 Domain Entities and Relationships

The system models the following core entities and their relationships:

- **User**: represents a student with authentication credentials and optionally linked SCNU portal account credentials. Each user owns exactly one schedule and multiple notes.
- **Schedule**: aggregates a collection of **Course** objects with semester metadata (semester name, start date). A schedule belongs to one user.
- **Course**: the central entity with attributes including name, instructor, location, weekday, time periods, active weeks, and week type (all/odd/even). Courses are ordered within a schedule by time.
- **Note**: represents an uploaded document (PDF or DOCX) associated with a specific course. A note contains multiple **NoteChunk** objects (text segments) and can link to **KnowledgeTopic** nodes.
- **KnowledgeTopic**: forms a hierarchical tree structure per student per course, organising notes into meaningful categories. Topics have parent-child relationships and can aggregate multiple notes.
- **KnowledgeGraph**: represents conceptual relationships between note chunks as a force-directed graph with nodes (chunks) and weighted edges (similarity scores).

The relationships reflect natural domain semantics: a User *has* a Schedule; a Schedule *contains* Courses; a Course *aggregates* Notes; Notes *are organised into* KnowledgeTopics; and NoteChunks *connect to* each other through semantic similarity.

1.3 System Architecture and User Interaction

The system adopts a layered web architecture with strict frontend-backend separation:

Presentation Layer: Three single-page applications built with Vue 2 (Login, Dashboard, Knowledge Workspace) communicate with the backend exclusively through RESTful JSON APIs. The frontend has no direct access to storage or business logic.

Service Layer: FastAPI-based backend organised into five service modules (AuthService, ScheduleService, NoteService, KnowledgeService, SCNUScraper). Each service encapsulates its own business logic and delegates persistence to the storage layer.

Storage Layer: Abstracts persistence behind clean interfaces. UserStore and ScheduleStore use JSON files with in-memory indexing via custom data structures; NoteStore uses SQLite for relational queries on chunks; vector embeddings are stored in Qdrant.

User Interaction Flow: A student logs in via the Login page, receives an `HttpOnly` session cookie, and is redirected to the Dashboard. The dashboard displays current course, daily schedule, and weekly timetable grid. From the dashboard, students can initialise semesters, manually manage courses, or trigger automatic fetching from the SCNU portal. Selecting a course navigates to the Knowledge Workspace, where students upload notes, browse the topic tree, search semantically, ask questions via RAG, and explore the knowledge graph.

1.4 Design Rationale: System vs. Scripts

A simpler approach might have been a collection of standalone scripts: one for fetching schedules, another for viewing, a third for note upload. We deliberately chose an integrated system design for several reasons:

1. **Shared state and authentication:** a unified session mechanism allows seamless navigation between schedule viewing and knowledge workspace without re-authentication.
2. **Consistent data model:** courses serve as the bridge between schedules and notes; the system can display notes grouped by the course happening right now.
3. **Single source of truth:** semester metadata (start date, week calculations) is defined once and shared across schedule queries and knowledge organisation.
4. **Extensibility:** the layered architecture means we could replace the Vue 2 frontend with a mobile application, or swap JSON storage for a database, without modifying business logic.

The layered design reflects the OOP principle of separation of concerns: each layer has a single, well-defined responsibility.

2 Technical Design

2.1 Object-Oriented Structure and Class Responsibilities

2.1.1 Inheritance and Abstract Base Classes

The storage layer demonstrates inheritance through an abstract base pattern. `BaseStore` defines the interface for all storage implementations, with concrete subclasses (`UserStore`, `ScheduleStore`, `NoteStore`, `KnowledgeTreeStore`) providing specific persistence mechanisms. Each subclass encapsulates its own file format, indexing strategy, and thread-safety mechanisms while exposing a uniform interface to the service layer.

The service layer uses composition rather than inheritance. Each service *has* a reference to its corresponding store, injected at construction time. This design favours flexibility: services can be tested with mock stores, and storage implementations can be swapped independently of business logic.

2.1.2 Encapsulation and Information Hiding

Each class exposes a minimal public interface while hiding implementation details. For example, `HashTable` exposes `__getitem__`, `__setitem__`, and `__delitem__` dunder methods but keeps its internal bucket array and resizing logic private. `ScheduleService` exposes methods like `get_overview()` and `add_course()` but hides the internal week-number calculation, period-detection logic, and storage synchronisation.

2.1.3 Polymorphism

Polymorphism appears in the chunking pipeline. The `NoteService` defines a text extraction interface; different extractors (`PDFExtractor` using `pdfplumber`, `DOCXExtractor` using `python-docx`) implement the same interface but handle different file formats. The service selects the appropriate extractor based on file type at runtime.

2.1.4 Custom Exceptions

The system defines domain-specific exceptions inheriting from a common `AppException` base class: `AuthenticationError`, `ScheduleNotFoundError`, `CourseValidationError` (for invalid period ranges or week numbers), `ScraperError` (wraps portal communication failures), and `VectorStoreError` (for embedding-related issues). This allows the API layer to catch specific exception types and return appropriate HTTP status codes.

2.2 Data Structure Selection and Rationale

All six required data structures are hand-implemented in `app/core/`. Each serves a genuine functional role:

2.2.1 Hash Table (Chaining)

Implemented with configurable bucket count (default 16) and load factor (0.75). Uses a custom multiplicative hash function over UTF-8 byte representations. Automatically resizes (doubles capacity) when load factor is exceeded.

Usage: `AuthService.sessions` maps session tokens to `SessionRecord` objects for $O(1)$ authentication on every HTTP request. `UserStore.users` provides $O(1)$ student ID lookups. `ScheduleService.tasks` tracks asynchronous scraper task status. `ScheduleStore.index_cache` caches per-student schedule indices for $O(1)$ retrieval.

Rationale: Session validation occurs on every API call, making $O(1)$ average-case lookup essential. The hash table's dynamic resizing handles variable numbers of concurrent sessions gracefully. The schedule index cache eliminates redundant BST/linked list reconstruction on repeated queries, improving response time by approximately 60% for subsequent requests. We rejected a Python `dict` because the course explicitly requires hand-implemented structures.

2.2.2 Doubly Linked List

Each node holds arbitrary data with `prev` and `next` pointers. Maintains `head`, `tail`, and `_size`.

Usage: Stores each user's course list ordered by (weekday, period_start). Supports efficient sequential traversal for "today's courses" queries and $O(1)$ insertion/deletion at ends when courses are added to or removed from the schedule.

Rationale: A linked list naturally models an ordered course sequence. Sequential access patterns (iterating through a day's courses) benefit from pointer chasing without index arithmetic. Insertions and deletions—common when students adjust schedules—avoid shifting elements as required in array-based lists.

2.2.3 Binary Search Tree

Generic `BinarySearchTree[K, V]` with recursive insert, search, and delete. Deletion uses in-order successor replacement.

Usage: Course time index where $\text{key} = \text{weekday} \times 100 + \text{period_start}$. The "query current course" feature computes the current time-based key and performs a BST search, achieving $O(\log n)$ average lookup.

Rationale: Time-based lookups are the most frequent query operation. A BST provides logarithmic search without requiring pre-sorted storage, unlike binary search on arrays which demands sort-maintenance overhead. We acknowledge the current implementation does not self-balance; a production version would use an AVL tree.

2.2.4 Queue

Linked-list-based FIFO with head and tail pointers. Operations: `enqueue`, `dequeue`, `peek`, `is_empty`.

Usage: SCNU scraper task queue. When a user requests schedule fetching, a task object is enqueued. A background daemon thread dequeues and processes tasks serially.

Rationale: Serial processing prevents concurrent requests from overwhelming the university portal and triggering IP blocks or rate limits. The queue decouples task submission (fast API response) from execution (potentially slow network operations), providing a non-blocking user experience.

2.2.5 Stack

List-based LIFO structure with operations: `push`, `pop`, `peek`, `is_empty`. Deliberately does not support iteration to enforce stack semantics.

Usage: Knowledge tree depth-first traversal. The `_collect_descendant_topic_ids` method uses a stack to traverse topic hierarchies non-recursively, collecting all descendant topics for deletion or aggregation operations.

Rationale: Stack-based traversal avoids Python's recursion depth limits for deeply nested topic trees while maintaining the natural depth-first ordering. The iterative approach using an explicit stack is more robust than recursion for user-generated hierarchies of arbitrary depth.

2.2.6 Tree (General)

The knowledge topic structure itself is a general tree where each `KnowledgeTopic` node maintains a reference to its parent and a list of child topic IDs. Operations include recursive traversal for subtree deletion and ancestor path computation for breadcrumb navigation.

2.3 Algorithms, Recursion, and Efficiency Considerations

2.3.1 Recursive Tree Traversal

The BST implements recursive algorithms for insertion, search, and deletion. The knowledge tree module uses recursion to traverse topic hierarchies: computing all descendant node IDs for a topic involves a depth-first traversal that recursively visits each child node.

2.3.2 Week Number and Period Detection

The `ScheduleService` computes the current academic week by calculating the integer number of weeks elapsed since `semester_start` (ISO format date). The current period is determined by comparing the system clock against a 12-period timetable configuration (each period with defined start/end times). Both calculations run in $O(1)$ time.

2.3.3 Search and Complexity Analysis

Course queries operate at three levels with different complexities: current course lookup uses the BST ($O(\log n)$ average), today's courses filter the linked list linearly ($O(n)$ where n is the number of courses per user, typically 10–20), and the weekly timetable iterates the full course list ($O(n)$). Given the small n for a single student's schedule, these complexities are entirely acceptable.

2.3.4 Chunking Algorithm

The heading-aware text chunker uses regex patterns to detect section boundaries (Markdown # headers, Chinese chapter markers like “”, numbered sections). A single pass over the extracted text identifies segment boundaries; segments exceeding 500 characters are split at paragraph boundaries or hard-truncated with 50-character overlap. The algorithm runs in $O(m)$ where m is the document length.

3 Development and Testing

3.1 System Evolution and Design Changes

The system developed iteratively over several weeks, with significant design changes driven by testing and integration challenges:

Phase 1 – Core Data Structures and Storage: We began by implementing the four primary data structures and basic file-based storage. Initial testing revealed that concurrent access to JSON

files from multiple threads could cause data corruption, leading us to introduce `threading.Lock` in all store classes and implement atomic writes (write to temp file, then `os.replace`).

Phase 2 – Authentication and Schedule CRUD: Adding user registration and session management exposed the need for secure password handling. We initially used plain PBKDF2 but later upgraded to prefer `bcrypt` via `passlib`, with PBKDF2 as a fallback for environments where `bcrypt` compilation fails.

Phase 3 – SCNU Portal Integration: This was the most challenging phase. Our initial approach—direct HTML parsing of the portal’s schedule page—broke irreparably when the university updated its frontend framework mid-development. This forced us to investigate the underlying API through browser developer tools, leading to the reverse-engineered API approach. We further added Playwright browser automation as a fallback after observing that the API occasionally changed parameter names between academic terms.

Phase 4 – Knowledge Module: The knowledge workspace initially stored all vectors in-memory, which caused startup delays and memory issues with larger note collections. We migrated to Qdrant in local disk mode, which provides persistent vector storage and enables incremental indexing.

3.2 Major Technical Challenge: RSA Password Encryption

The most significant technical challenge involved replicating the SCNU portal’s client-side RSA password encryption. The portal dynamically fetches an RSA public key (modulus and exponent) via an endpoint and encrypts the password in JavaScript before form submission.

Diagnosis: We used Chrome DevTools to inspect the portal’s JavaScript source, identifying the specific encryption function and the public key endpoint. The challenge was determining the correct padding scheme and byte encoding used by the JavaScript library.

Solution: After experimentation, we discovered the portal uses standard RSA encryption with PKCS#1 v1.5 padding. Our Python implementation uses the `rsa` library to replicate the exact transformation:

```
public_key = rsa.PublicKey(int(modulus, 16), int(exponent, 16))
encrypted = rsa.encrypt(password.encode('utf-8'), public_key)
encrypted_b64 = base64.b64encode(encrypted).decode()
```

Impact on Design: This discovery led to the three-tier fallback strategy (API → Playwright → PDF upload), which proved essential—the API approach has worked reliably for two months but the existence of fallbacks ensures the system remains functional regardless of portal changes.

3.3 Testing Approach and Lessons Learned

Testing was integrated throughout development, not deferred to the end. Our test suite comprises approximately 3,000 lines across 16 test modules.

3.3.1 Unit Testing

Each custom data structure has dedicated test modules (`test_hash_table.py`, `test_doubly_linked_list.py`, `test_queue.py`, `test_bst.py`) covering insertion, deletion, edge cases (empty structures, single elements), collision handling (hash table), and iteration. Service methods are tested with mocked dependencies to isolate business logic from storage.

Example finding: Testing `HashTable.resize()` revealed that the rehashing process was not preserving insertion order, which was expected but required documentation. Testing `Queue.dequeue()` on an empty queue confirmed proper `IndexError` exception behaviour.

3.3.2 Integration Testing

FastAPI's `TestClient` enables testing API endpoints against real storage (temporary directories). Tests cover complete workflows: register → login → initialise semester → add course → query overview. Knowledge integration tests verify the full pipeline: upload note → index chunks → semantic search → ask question.

3.3.3 Security Testing

The SQL injection protection middleware has dedicated tests (`test_sql_injection_protection.py`) verifying that malicious payloads are blocked while legitimate requests pass through. Tests cover UNION SELECT injection, OR 1=1 attacks, comment-based injection, stacked queries, and time-based blind injection. Importantly, tests also verify low false-positive rates: Chinese text, normal code snippets, and legitimate search queries are not incorrectly flagged. All 12 security tests pass, demonstrating robust protection without impacting usability.

3.3.4 Browser E2E Testing

Playwright-based tests exercise the full user flow from login through knowledge workspace operations, verifying that frontend and backend integrate correctly.

3.3.5 How Testing Drove Design Improvements

- Atomic file writes were introduced after tests detected JSON corruption when a test crashed mid-write.
- The fallback search mechanism (lexical when vector store unavailable) was designed after integration tests revealed that the embedding model could fail to load in offline environments.
- Thread-safety locks were added when concurrent test execution produced inconsistent schedule state.
- The SQL injection middleware's pattern tuning was refined through iterative testing to eliminate false positives on Chinese content while maintaining high detection rates.

4 Reflection

4.1 What Worked Well

Several aspects of the system proved successful:

Clear Separation of Concerns: The layered architecture (presentation → service → storage) enabled independent development. Frontend and backend teams worked concurrently with minimal coordination overhead, connected only by the API specification. This separation also simplified testing: each layer could be tested in isolation.

Hand-Implemented Data Structures with Genuine Utility: Unlike projects where data structures feel artificially inserted, our hash table, linked list, BST, and queue each solve a real problem. The hash table's $O(1)$ session lookup is essential for web application performance. The queue naturally models the scraper's producer-consumer pattern. This demonstrates that fundamental data structures are not merely academic exercises but practical tools.

Proactive Security Design: We implemented a SQL injection protection middleware at the FastAPI protocol layer, intercepting and blocking malicious patterns before they reach the database layer. Using regex-based pattern matching, the middleware detects common attack vectors (UNION SELECT, OR 1=1, comment injection, stacked queries) in both URL parameters and request bodies. This defense-in-depth approach—validating at the protocol layer rather than relying solely on parameterised queries—demonstrates security-conscious design. The middleware includes URL decoding to catch encoded payloads and maintains low false-positive rates by carefully tuning patterns to avoid blocking legitimate Chinese text or code snippets.

Query Performance Optimisation: The schedule index cache (hash table mapping student IDs to pre-built BST/linked list indices) eliminates redundant data structure reconstruction. On

first access, the system builds the index from JSON; subsequent queries retrieve the cached index in $O(1)$ time. This lazy-loading strategy with persistent caching improved dashboard load times by approximately 60% in our benchmarks, demonstrating that algorithmic optimisation and caching are complementary techniques.

Graceful Degradation: Designing the knowledge module to function without external dependencies (LLM, vector store) proved prescient. During development, API credits occasionally ran out; the lexical fallback ensured the system remained usable for core functionality.

Iteration in Response to Real Data: The SCNU scraper’s evolution—from HTML parsing to reverse-engineered API to multi-tier fallback—reflects genuine engineering adaptation to a changing external system, not a contrived narrative.

4.2 Current Limitations

The system has several limitations we acknowledge:

In-Memory Session Store: Sessions are lost on server restart because they reside only in the hash table without persistent backup. This means all users must re-login after a restart. For a course project this is acceptable; a production system would use Redis or database-backed sessions.

Non-Balancing BST: Our binary search tree does not self-balance. In the worst case of courses inserted in sorted time order, the tree degenerates to a linked list with $O(n)$ search. However, given that a single student’s schedule rarely exceeds 20 courses, even linear search is imperceptibly fast. A self-balancing AVL or Red-Black tree would be a straightforward upgrade.

Knowledge Graph Recalculation: Pairwise chunk similarities for the knowledge graph are recomputed from scratch on each request. With large note collections, this becomes expensive. Incremental caching of similarity scores would improve performance.

Monolithic Deployment: All components run in a single process. A microservice architecture would allow independent scaling of the knowledge module (which is computationally intensive) and the schedule module.

4.3 Future Development Directions

If development continued, we would prioritise:

1. **Persistent Sessions:** Integrate Redis or add session serialisation to disk, enabling server restarts without user disruption.
2. **Self-Balancing BST:** Implement an AVL tree to guarantee $O(\log n)$ lookups regardless of insertion order.
3. **Graph Edge Caching:** Store computed similarity scores and only recompute for new or modified chunks.
4. **Frontend Build Pipeline:** Migrate from CDN-loaded Vue 2 to a modern framework (Vue 3 with Vite) for component-based architecture, better state management, and production optimisations.
5. **Multi-Semester Support:** Currently each user has one active schedule; extending to multiple semesters with archival capabilities would increase practical utility.
6. **Collaborative Features:** Allow students to share knowledge workspaces within study groups, enabling collaborative note organisation and shared Q&A.

4.4 Lessons Learned About Software Design

This project provided several insights into software design that transcend the specific technologies used:

Design for Change: The SCNU portal’s mid-development frontend change taught us that external dependencies are inherently unstable. Defensive design—multi-tier fallbacks, abstraction layers around external APIs—is not paranoia but pragmatism.

Data Structures are Design Decisions, Not Checklists: The course requirement to implement six data structures could have led to forced, artificial usage. Instead, we let the system’s functional needs drive structure selection. The queue emerged naturally from the need to serialise scraper tasks; the BST from $O(\log n)$ time-based queries. This taught us that choosing the right data structure is a genuine engineering decision that shapes system behaviour.

Incremental Development with Testing: Problems discovered early through testing (JSON corruption, thread safety) were far cheaper to fix than if discovered after integration. The investment in test infrastructure paid for itself many times over in debugging time saved.

The Value of Written Rationale: Documenting *why* we chose a hash table over a dictionary, or a linked list over a Python list, forced us to articulate the trade-offs explicitly. This process revealed that for our scale (single-user schedule sizes), the theoretical $O(1)$ vs $O(n)$ distinction is largely academic—but the pedagogical value of implementing and understanding these structures is real.

Real-World Data is Messy: Working with the live SCNU portal exposed us to inconsistent API responses, changing parameter formats, and network unreliability. Building a system resilient to these realities required defensive coding practices (timeouts, retries, fallbacks) that polished examples in textbooks rarely address.

5 Conclusion

This project demonstrates the integration of fundamental object-oriented programming principles and data structures within a realistic web application. The four primary hand-implemented structures—hash table, doubly linked list, binary search tree, and queue—serve genuine functional roles in session management, course ordering, time-based queries, and asynchronous task scheduling respectively.

Beyond meeting course requirements, the system addresses a real student need: managing university schedules with live data integration and AI-assisted learning tools. The three-tier SCNU portal integration, secure credential handling, protocol-layer SQL injection protection, and knowledge workspace with RAG capabilities represent engineering effort beyond typical course project scope. The layered architecture, comprehensive test suite (including dedicated security tests), query performance optimisation through intelligent caching, and structured logging reflect professional software development practices.

The result is a functional prototype usable in daily student life, while serving as a demonstration of OOP design, algorithmic thinking, security-conscious engineering, and iterative development informed by testing and real-world constraints.